

PHISHING DETECTION USING BEHAVIOURAL CUES IN BROWSER INTERACTION

A PROJECT REPORT

Submitted by

PRIYADHARSHINI S (922222104037)

SANDHIYA K (922222104040)

THANGAMAYIL R (922222104048)

in partial fulfillment for the award of the degree

Of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

THENI KAMMAVAR SANGAM COLLEGE OF TECHNOLOGY

THENI

ANNA UNIVERSITY:: CHENNAI 600 025

APRIL 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**PHISHING DETECTION CUES IN BROWSER INTERACTION**” is the Bonafide work of **PRIYADHARSHINI S (922222104037), SANDHIYA K (922222104040), THANGAMAYIL R (922222104048)** ,who carried out the project work under my supervision.

SIGNATURE

Mr.D. ANANTH

HEAD OF THE DEPARTMENT

Computer Science and Engineering

Theni Kammavar Sangam College of
Technology

Theni - 625534.

SIGNATURE

Mr . K. AYYAPPARAJA

SUPERVISOR

Assistant Professor

Computer Science and Engineering

Theni Kammavar Sangam College of
Technology

Theni - 625534.

Submitted for the VIVA - VOCE Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

This research presents an innovative phishing detection system that leverages user behavioral analytics and machine learning to identify malicious websites. Traditional phishing detection methods primarily focus on URL analysis, content inspection, and blacklisting, which often fail against sophisticated attacks. Our proposed system introduces a novel approach by analyzing real-time user interaction patterns including mouse movements, click behavior, keystroke dynamics, scrolling activity, and session duration to distinguish between legitimate and phishing websites. By implementing a Random Forest Classifier trained on comprehensive behavioral datasets, the system achieves high accuracy in detecting phishing attempts. The web-based application, built with Flask framework, provides both manual input analysis and automatic real-time detection capabilities, offering a proactive defense mechanism against evolving phishing threats. This behavioral-based approach represents a significant advancement in phishing detection, moving beyond static analysis to dynamic user interaction monitoring.

LIST OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF ABBREVIATIONS	iii
1	INTRODUCTION	
	1.1 Introduction	1
	1.2 Background of Study	2
2	LITERATURE SURVEY	
	2.1 Introduction	4
	2.2 Survey	11
3	METHODOLOGY	
	3.1 Work plan	15
	3.1.1 Problem Definition	
	3.1.2 Data Collection	
	3.1.3 Data Preprocessing	
	3.1.4 Model Selection	
	3.2 CNN	21
	3.3 LSTM	25

4	SYSTEM DESIGN	
	4.1 System Architecture	31
	4.2 Data Flow Diagram	34
	4.2.1 Use Case Diagram	35
	4.2.2 Class Diagram	36
5	MODULES	
	5.1 Data collection and preprocessing module	37
	5.2 Machine Learning Training Module	37
	5.3 Real-time Detection Engine Module	38
	5.4 Web Application Interface Module	38
	5.5 Database Management and Analytics Module	39
6	CONCLUSION	46
	APPENDIX	47
	REFERENCES	49

LIST OF FIGURES

S. No	NAME	PAGE NO
1	System architecture	31
2	Data Flow Diagram	34
3	Use Case Diagram	35
4	Class Diagram	36

LIST OF ABBREVIATIONS

CNN	-	Convolutional Neural Network
XSS	-	Cross-Site Scripting
URL	-	Uniform Resource Locator
APWG	-	Anti-Phishing Working Group
RNN	-	Recurrent Neural Network
LSTM	-	Long Short Term Memory
GRU	-	Gated Recurrent Unit
IQR	-	Inter Quartile Range
SMOTE	-	Synthetic Minority Over sampling Technique

CHAPTER - 1

1.1 INTRODUCTION

In today's increasingly interconnected world, web applications have become indispensable tools for businesses, governments, and individuals. From e-commerce platforms and social networking sites to online banking systems and educational portals, these applications offer unparalleled convenience, accessibility, and scalability, fundamentally transforming the way people communicate, transact, and consume information. However, this digital transformation has also created fertile ground for cyber threats, as the very technologies that facilitate seamless online experiences simultaneously expose sensitive data and critical systems to malicious actors. As organizations and users become more dependent on web applications, the potential attack surface expands, making security a paramount concern. Web vulnerabilities such as SQL injection, cross-site scripting (XSS), broken authentication, insecure deserialization, and misconfigured security settings represent some of the most prevalent and dangerous pathways for attackers to exploit. If these weaknesses are left unaddressed, they can result in unauthorized data breaches, operational disruptions, reputational damage, and significant financial losses. Understanding and mitigating these vulnerabilities is therefore essential not only for protecting digital assets but also for ensuring the continued trust and safety of users in an increasingly online world. Python, with its versatile frameworks and libraries, has emerged as a valuable tool in identifying, analyzing, and defending against these web vulnerabilities, providing developers and security professionals with practical means to enhance the resilience of web applications.

1.2 BACKGROUND OF STUDY

1. Phishing Overview

Phishing is a type of cyberattack where attackers design fake websites that closely resemble legitimate ones to steal sensitive information such as login credentials and financial details. These sites imitate trusted platforms like banks, social media, and email services, deceiving users into entering their data.

2. Limitations of Traditional Detection Methods

Conventional phishing detection techniques include URL analysis, content inspection, and maintaining blacklists of known malicious sites. However, these approaches have several limitations. Attackers frequently generate new URLs that are not yet blacklisted, advanced phishing sites can evade content-based detection, and zero-day attacks often go unnoticed. Additionally, these methods struggle to identify highly targeted spear phishing attacks.

3. Importance of Behavioral Analysis

User behavior plays a crucial role in identifying phishing attempts. On legitimate websites, users tend to interact naturally and confidently. In contrast, phishing sites often trigger subtle behavioral changes such as hesitant mouse movements, irregular clicking patterns, unusual typing speed, shorter browsing sessions, and abnormal scrolling behavior.

4. Role of Machine Learning in Cybersecurity

Machine learning has significantly improved cybersecurity by enabling systems to learn from data patterns instead of relying solely on predefined rules. It allows real-time anomaly detection and adapts to emerging threats. The Random Forest Classifier is particularly effective because it can handle complex behavioral data,

minimize overfitting, provide insights into important features, and achieve high accuracy using multiple decision trees.

5. Proposed Approach

This study integrates behavioral psychology with cybersecurity by collecting real-time user interaction data and applying machine learning techniques to detect phishing patterns. The system supports both manual and automated detection, offering a proactive defense strategy.

6. Significance of the Study

The proposed approach provides several benefits. It enables proactive detection of zero-day attacks, focuses on user behavior rather than only website features, adapts continuously to new data, and complements existing detection systems.

7. Research Gap

Most current solutions focus on analyzing website properties, while very few consider user behavior. This study addresses that gap by emphasizing that user interaction patterns are strong indicators of a website's legitimacy something that is difficult for attackers to replicate.

CHAPTER - 2

LITERATURE SURVEY

2.1 Introduction

The rapid expansion of internet-based services has fundamentally transformed how individuals and organizations conduct daily activities, from banking and shopping to communication and social networking. This digital transformation, while bringing unprecedented convenience, has also created new avenues for cybercriminals to exploit unsuspecting users. Among the most pervasive and damaging cyber threats in the digital landscape is phishing a form of social engineering attack where malicious actors create fraudulent websites, emails, or messages that mimic legitimate entities to deceive users into revealing sensitive information such as login credentials, credit card numbers, and personal identification data (Gupta et al., 2022).

Phishing attacks have evolved significantly since their first documented appearance in the mid-1990s against America Online (AOL). What began as relatively crude attempts has matured into sophisticated, targeted campaigns that leverage advanced social engineering techniques and professional-looking web designs. According to the Anti-Phishing Working Group (APWG), the first quarter of 2024 recorded over 1.2 million unique phishing attacks globally, marking a continued upward trend that shows no signs of abating (APWG, 2024). The financial impact is equally staggering, with the FBI's Internet Crime Complaint Center reporting losses exceeding \$10 billion annually due to phishing and related cyber fraud (IC3, 2023).

The success of phishing attacks stems from their exploitation of human psychology rather than technical vulnerabilities. Attackers craft scenarios that evoke urgency,

fear, curiosity, or trust emotions that override rational decision-making and cause users to bypass normal security precautions. This human-centric nature makes phishing particularly challenging to combat through technical measures alone, as the weakest link in the security chain often remains the user themselves (Lastdrager, 2014).

Evolution of Phishing Detection Techniques

The academic and industrial communities have responded to the phishing threat with a diverse array of detection methodologies that have evolved in sophistication alongside the attacks themselves. Early detection systems relied heavily on blacklisting approaches, where known malicious URLs were compiled into databases that browsers could reference to warn users (Sheng et al., 2009). Services like Google Safe Browsing and Phish Tank exemplify this approach, maintaining extensive repositories of reported phishing sites. However, the fundamental limitation of blacklisting is its reactive nature sites are only added after they have been identified as malicious, leaving a critical window of opportunity for attackers during the early hours of a campaign.

Heuristic-based methods emerged as a proactive alternative, analyzing website characteristics such as URL structure, domain age, page content, and visual similarity to legitimate sites (Zhang et al., 2007). These systems assign risk scores based on predefined rules and feature sets. For instance, Cantina and Cantina+ developed by Xiang et al. (2011) employed content-based features combined with search engine rankings to classify phishing pages. While heuristics improved detection rates for zero-day attacks, they remained vulnerable to evasion techniques as attackers learned to mimic legitimate site characteristics more convincingly.

The integration of machine learning represented a paradigm shift in phishing detection research. Unlike rule-based heuristics, machine learning algorithms could automatically learn complex patterns from large datasets without explicit programming of detection rules. Early studies by Basnet et al. (2008) demonstrated the feasibility of applying classification algorithms including Support Vector Machines (SVM), Naïve Bayes, and Decision Trees to URL-based features. These approaches achieved promising accuracy rates between 85% and 95% on benchmark datasets, establishing machine learning as a dominant research direction in the field.

Machine Learning Approaches to Phishing Detection

The machine learning landscape for phishing detection has expanded considerably, encompassing a wide range of algorithms and feature engineering strategies. Random Forest, an ensemble learning method combining multiple decision trees, has emerged as particularly effective due to its robustness against overfitting and ability to handle high-dimensional feature spaces (Abdelhamid et al., 2014). Studies comparing multiple classifiers on comprehensive phishing datasets found that Random Forest consistently outperformed alternatives including SVM and Neural Networks, achieving accuracy rates exceeding 97% when trained on hybrid feature sets combining URL characteristics, page content, and external metadata.

Deep learning approaches have gained traction in recent years, with convolutional neural networks (CNNs) and recurrent neural networks (RNNs) demonstrating remarkable capability in extracting hierarchical features from raw URL strings and HTML content. These architectures eliminate the need for manual feature engineering by learning relevant representations directly from data. Bahnsen et al. (2017) applied long short-term memory (LSTM) networks to URL character

sequences, achieving comparable performance to traditional feature-based methods while requiring less domain expertise for feature design.

Despite these advances, a critical limitation persists across most machine learning-based detection systems: they operate on static snapshots of website characteristics. Whether analyzing URLs, HTML content, or visual appearance, these approaches examine the website as an artifact rather than considering the dynamic interaction between user and site. This static orientation leaves systems blind to behavioral cues that may distinguish legitimate from malicious sites during actual user sessions.

Behavioral Analytics in Cybersecurity

The recognition that human behavior contains valuable security signals has spurred growing interest in behavioral analytics across various cybersecurity domains. Keystroke dynamics, for example, has been extensively studied as a biometric authentication mechanism, with research demonstrating that individuals exhibit unique and consistent typing patterns including dwell time, flight time, and error rates (Monrose & Rubin, 2000). Similarly, mouse dynamics analyzing movement trajectories, click patterns, and acceleration profiles has shown promise for continuous user authentication (Shen et al., 2013).

In the context of intrusion detection, behavioral analysis offers the advantage of detecting anomalies that static signatures would miss. The underlying premise is that legitimate and malicious activities produce distinguishable behavioral footprints. For instance, research by Eberle et al. (2010) applied graph-based anomaly detection to identify insider threats by analyzing deviations from normal user behavior patterns. These approaches share a common insight: behavior is inherently difficult to fake convincingly over time.

The application of behavioral analytics to phishing detection remains relatively unexplored compared to other security domains. However, emerging research suggests that user interaction patterns with websites contain valuable discriminatory information. When users encounter phishing sites, subtle behavioral changes may manifest hesitation before clicking, unusual navigation patterns, or abbreviated sessions that could serve as detection signals. Conversely, legitimate sites typically elicit more natural, fluid interactions reflecting user familiarity and comfort.

The specific behavioral features relevant to phishing detection encompass multiple dimensions of user-website interaction. Mouse movement characteristics, including velocity, acceleration, curvature, and click precision, provide rich information about user cognitive state and familiarity with the interface (Huang et al., 2011). Studies in human-computer interaction have established that uncertainty, confusion, or suspicion manifest in measurable changes to motor behavior phenomena that could potentially distinguish interactions with deceptive websites.

Clickstream analysis offers another behavioral dimension, capturing the sequence and timing of user actions during a session. Research by Padmanabhan et al. (2018) demonstrated that click patterns on malicious websites differ systematically from legitimate counterparts, with users exhibiting more erratic navigation and higher abandonment rates on phishing sites. Similarly, scrolling behavior, including scroll speed, pause patterns, and depth of exploration, may reflect user engagement and perceived legitimacy.

Keystroke dynamics extends beyond simple typing patterns to include form-filling behavior on web pages. When entering sensitive information on suspected phishing sites, users may type more slowly, pause more frequently, or exhibit different error

correction patterns compared to trusted environments (Roth et al., 2013). Session duration metrics provide a high-level behavioral signal, with shorter sessions potentially indicating user discomfort or early abandonment upon recognizing deception.

Research Gap and Proposed Contribution

Despite the theoretical promise of behavioral features for phishing detection, systematic investigation of this approach remains limited. Existing literature predominantly focuses on static website characteristics, with behavioral considerations confined to user education and awareness training rather than automated detection. The few studies that incorporate user behavior do so indirectly, such as analyzing traffic patterns rather than individual interaction dynamics.

This research addresses this gap by proposing a novel phishing detection framework that integrates real-time user behavioral analytics with machine learning classification. The core hypothesis is that interaction patterns mouse movements, click behavior, keystroke dynamics, scrolling activity, and session duration contain discriminative information that can distinguish legitimate from phishing websites with high accuracy. By implementing a Random Forest Classifier trained on comprehensive behavioral datasets, the system aims to achieve robust detection performance while remaining adaptable to evolving attack strategies.

The Flask-based web application developed in this research provides both manual input analysis and automatic real-time detection, demonstrating the practical feasibility of behavioral monitoring in production environments. This dual-mode operation allows users to submit suspicious URLs for analysis while

simultaneously benefiting from passive monitoring of their own browsing sessions. The result is a proactive defense mechanism that complements existing detection approaches by adding a behavioral dimension previously unexploited in phishing detection.

Organization of the Paper

The remainder of this paper is organized as follows. Section 2 presents the system architecture and methodology, detailing the feature extraction process, Random Forest classifier implementation, and web application design. Section 3 describes the experimental setup, including dataset collection, preprocessing procedures, and evaluation metrics. Section 4 presents experimental results and analysis, comparing the proposed behavioral approach with traditional methods. Section 5 discusses implications, limitations, and future research directions. Section 6 concludes the paper with a summary of contributions and findings.

2.2 Survey

1. Survey and comparative analysis of phishing detection techniques: current trends, challenges, and future directions

Authors: Ashvini Jadhav, Pankaj R. Chandre

Published in: IAES International Journal of Artificial Intelligence (IJ-AI), 2025

This comprehensive review examines various phishing detection methods, categorizing them by application (email, web server, mail server, or browser-based). The authors provide a comparative evaluation of signature-based, heuristic-based, machine learning-based, and behavior-based techniques, analyzing their respective advantages and disadvantages. The study highlights that machine learning algorithms, particularly Random Forest and Support Vector Machines, consistently deliver high accuracy for phishing detection. The paper also explores recent developments including ML-powered social media exploration and real-time website analysis, concluding that hybrid approaches combining multiple techniques offer stronger defense mechanisms against evolving phishing threat .

2. Real-Time Phishing Detection and User Education Using Machine Learning

Authors: Hajira Sultana, Aaileen Sarwar Syed

Published in: Zenodo (Preprint), June 2025

This paper introduces a novel real-time phishing detection system that combines technical detection with user education. The system employs Natural Language Processing techniques including sentiment analysis and TF-IDF vectorization, coupled with a Random Forest classifier to accurately identify phishing emails.

When suspicious emails are detected, the system provides immediate alerts through a graphical interface along with helpful safety tips, effectively training users to recognize phishing indicators. The dual approach of protection and education aims to strengthen the "human firewall" by reducing phishing success rates while increasing long-term user awareness. Experimental results demonstrate the system's effectiveness in distinguishing phishing emails from legitimate one .

3. Deep Learning-Driven User Legitimacy Prediction Using Keystroke and Mouse Behavioral Dynamics

Authors: Not specified in abstract (affiliated with Nile University, Egypt)

Published in: 2024 International Conference on Computer and Applications (ICCA), IEEE

This research presents a comprehensive comparison of traditional machine learning methods (Random Forest, KNN, Logistic Regression, XG Boost) against deep learning models (Deep Feedforward Neural Networks and LSTM) for continuous authentication based on behavioral dynamics. Using both keystroke and mouse dynamics datasets, the study found that LSTM models consistently outperformed other approaches, achieving 88% accuracy on keystroke dynamics data. The paper emphasizes the importance of decision time for real-time authentication systems and highlights the advantage of deep learning models in handling sequential behavioral data, while noting that traditional models showed slower response times and lower accuracy .

4. A Comparative Study of Machine Learning and Neural Network Models for Phishing Detection

Authors: Dimitar Rangelov, Radoslav Miltchev, Evgeni Genchev

Published in: 5th International Conference on Data Information in Online Environments (DIONE 2024), Springer, August 2025

This study provides a comprehensive comparison of traditional machine learning models (Naive Bayes, Logistic Regression, Decision Trees, Random Forests) against advanced neural network architectures (RNN, LSTM, GRU) for phishing email detection. The authors analyze trade-offs between model complexity, computational cost, and accuracy, focusing on scalability to real-world attacks. Results indicate that while neural networks, particularly LSTM and GRU, effectively capture complex patterns in sequential email data, simpler ML models such as Stochastic Gradient Descent achieve competitive accuracy with significantly lower computational overhead. This balance makes ML models particularly suitable for large-scale, real-time phishing detection systems .

5. Multi-Modal Adversarial Activity Detection Using Keyboard and Mouse Dynamics

Authors: Not specified (SJSU Scholar Works)

Published in: San José State University Faculty Research, 2025

This research presents a multi-modal approach to behavioral biometric classification by fusing keystroke and mouse activity data for detecting adversarial behavior. The authors introduce a temporal alignment framework that segments and synchronizes fixed-size windows of keyboard input with corresponding mouse movement sequences, preserving temporal relationships between modalities. Using deep learning models including CNNs and LSTMs, the system classifies user sessions as benign or adversarial. Results demonstrate that integrating keyboard and mouse modalities significantly improves classification performance over modal systems, achieving up to 99% accuracy on a dataset of 100 users evenly

split between benign and adversarial behavior. This work highlights the value of multi-modal, non-intrusive behavioral biometrics for enhancing threat and intrusion detection systems .

These five papers collectively cover your key research areas: machine learning for phishing detection (Jadhav & Chandre; Rangelov et al.), user behavioral analytics (Nile University; SJSU), and real-time detection with user education (Sultana & Syed).

CHAPTER 3

METHODOLOGY

3.1 Work Plan

The digital landscape has witnessed an exponential increase in phishing attacks, with cybercriminals employing increasingly sophisticated techniques to deceive users and compromise sensitive information. Traditional phishing detection methods, including blacklisting, URL analysis, and content-based filtering, have demonstrated significant limitations in addressing modern threats. These conventional approaches operate on static features that attackers can easily manipulate or bypass, creating an urgent need for more robust and adaptive detection mechanisms.

The core problem addressed in this research is the detection gap that exists between static analysis techniques and the dynamic, evolving nature of phishing attacks. Current systems fail to capture the behavioral dimension of user interaction with websites a dimension that holds significant promise for distinguishing legitimate sites from malicious ones. When users interact with phishing websites, subtle behavioral changes occur: hesitation in mouse movements, altered typing patterns, unusual click behaviors, and modified scrolling activity. These behavioral cues, if properly captured and analyzed, could serve as powerful discriminative features for phishing detection.

This research identifies several critical gaps in existing literature and practice:

First, existing phishing detection systems predominantly focus on website-centric features URL structure, HTML content, visual similarity while completely overlooking user-centric behavioral signals.

Second, traditional machine learning approaches to phishing detection treat each website visit as an isolated event, failing to capture the temporal dynamics of user interaction. The sequence and timing of user actions contain rich information about user comfort, familiarity, and suspicion that static feature sets cannot capture.

Third, real-time detection capabilities remain limited in existing systems, with many approaches operating in batch mode or requiring significant computational resources that preclude real-time deployment. This limitation is particularly problematic given that phishing attacks succeed within minutes of user exposure.

Fourth, the integration of multiple behavioral modalities mouse movements, keystroke dynamics, click patterns, scrolling behavior, and session characteristics has not been systematically explored for phishing detection. Each modality captures different aspects of user behavior, and their fusion could potentially yield detection accuracy exceeding any single modality.

3.1.1 Problem Definition

The problem identification phase establishes that effective phishing detection requires a paradigm shift from static, website-centric analysis to dynamic, user-centric behavioral monitoring. This shift necessitates the development of novel feature extraction techniques, appropriate machine learning architectures, and real-time processing capabilities all of which form the core objectives of this research.

3.1.2 Data Collection

The success of any machine learning-based detection system depends fundamentally on the quality, quantity, and representativeness of the training data. For this research, data collection encompasses multiple dimensions: behavioral

data from user interactions with both legitimate and phishing websites, and corresponding ground truth labels for model training and evaluation.

Behavioral Data Collection Framework:

A custom data collection application was developed using Python and the Flask framework to capture comprehensive user interaction data. The application embeds JavaScript tracking code that records the following behavioral modalities in real-time

1. Mouse Movement Data : Mouse movement data includes X and Y coordinates sampled at 50ms intervals, along with movement velocity and acceleration to understand how the cursor travels across the screen. It also captures trajectory curvature and deviation metrics, providing insight into user intent and precision. Additional details such as click coordinates, pressure characteristics, double-click patterns and timing, as well as mouse wheel scroll events and speed, contribute to a comprehensive understanding of pointer-based interactions.

2. Keystroke Dynamics : Keystroke dynamics involve recording key press and release timestamps to analyse typing behaviour. This includes dwell time, which measures how long a key is held down, and flight time, which represents the interval between releasing one key and pressing the next. It also considers typing speed and rhythm patterns, error rates, and correction behaviour such as the use of backspace or delete keys, along with patterns observed during form field completion.

3. Clickstream Behaviour : Clickstream behaviour focuses on how users interact with clickable elements on a page by tracking click coordinates and target elements. It examines click timing, inter-click intervals, and click sequence

patterns to identify navigation habits. Additionally, it includes right-click and context menu usage, as well as user actions related to form submission behaviour.

4. Scrolling Activity : Scrolling activity captures how users navigate through page content by tracking scroll start and end positions, as well as scroll speed and acceleration. It also includes pause duration at different positions on the page, total scroll distance, and depth of exploration. Patterns of upward and downward scrolling further help in understanding user reading and browsing behaviour.

5. Session Characteristics : Session characteristics provide an overview of the entire user interaction session, including total session duration and page navigation patterns. It also measures the time spent on different sections of a page, identifies bounce rate indicators, and analyse abandonment behaviour to better understand user engagement and drop-off points.

3.1.3 Data Preprocessing

Raw behavioral data collected from user interactions requires extensive preprocessing before it can be effectively used for machine learning model training. The preprocessing pipeline addresses multiple challenges inherent in behavioral data, including noise, missing values, temporal misalignment, and feature normalization.

Noise Reduction and Filtering:

Behavioral data inherently contains noise from various sources sensor inaccuracies, network latency, user distractions, and environmental factors. A multi-stage filtering approach was implemented:

1. Outlier Detection and Removal: Statistical methods including Z-score analysis and interquartile range (IQR) filtering identify and remove extreme values that

likely represent data collection artifacts rather than genuine behavioral signals. For mouse movement data, positions outside screen boundaries or exhibiting impossible velocities are filtered.

2. Smoothing Filters: Golay filters are applied to mouse trajectory data to reduce high-frequency noise while preserving essential movement characteristics. This smoothing preserves the shape and dynamics of movements while eliminating sensor jitter.

3. Temporal Alignment: Different behavioral modalities operate at different sampling frequencies. Mouse movements are sampled at 50ms intervals, while keystroke events occur sporadically. A temporal alignment framework synchronizes these heterogeneous data streams into unified time windows of fixed duration (e.g., 5-second segments).

3.1.4 Model Selection

The selection of appropriate machine learning models for behavioral-based phishing detection involves careful consideration of multiple factors including the nature of behavioral data, temporal dependencies, computational requirements for real-time deployment, and interpretability needs. This research evaluates and ultimately selects two complementary deep learning architectures: Convolutional Neural Networks (CNN) for spatial feature extraction from behavioral patterns, and Long Short-Term Memory (LSTM) networks for capturing temporal dependencies in user interaction sequences.

Model Selection Criteria:

1. Temporal Modeling Capability: Behavioral data inherently contains sequential dependencies current actions influence and are influenced by previous actions. Models must capture these temporal relationships.
2. Spatial Pattern Recognition: Mouse movements and click distributions exhibit spatial patterns that distinguish legitimate from phishing interactions. Models should extract these spatial features effectively.
3. Real-time Processing: Selected models must support inference speeds compatible with real-time deployment, processing user interactions with minimal latency.
4. Multi-modal Integration: The architecture should accommodate multiple behavioral modalities, potentially through ensemble or fusion strategies.
5. Interpretability: For security applications, model interpretability is crucial understanding why a particular interaction was flagged as suspicious enables system improvement and user trust.

Candidate Model Evaluation:

Initial experimentation evaluated multiple architectures:

Random Forest: Strong baseline performance, excellent interpretability, but limited temporal modeling

Support Vector Machines: Good for high-dimensional feature spaces, but struggles with sequence data

CNN: Excellent for spatial pattern extraction, moderate temporal modeling through 1D convolutions

LSTM: Superior temporal modeling, captures long-range dependencies

3.2 Convolutional Neural Networks (CNN) for Behavioral Pattern Extraction

3.2.1 Theoretical Foundations of CNNs

Convolutional Neural Networks represent a specialized class of deep learning architectures designed to process data with grid-like topology. Originally developed for image processing tasks, CNNs have demonstrated remarkable effectiveness across diverse domains including time series analysis, natural language processing, and behavioral pattern recognition. The fundamental innovation of CNNs lies in their use of convolutional operations that preserve spatial relationships while learning hierarchical features through local connectivity and weight sharing.

Local Connectivity: Each neuron connects only to a local region of the input, enabling the network to learn local patterns such as specific mouse movement motifs or typing rhythm segments that characterize user behavior.

Weight Sharing: The same kernel weights are applied across all spatial locations, dramatically reducing parameter count and enabling translation invariance. This property ensures that a particular behavioral pattern is recognized regardless of where it occurs in the interaction sequence.

Hierarchical Feature Learning: Multiple convolutional layers learn increasingly abstract representations, from low-level features (e.g., specific movement directions) to high-level behavioral signatures (e.g., patterns indicating hesitation or suspicion).

3.2.2 CNN Architecture for Behavioral Data

The CNN architecture designed for this research operates on structured representations of user behavior. Rather than processing raw mouse coordinates or

keystroke timestamps directly, behavioral data is first transformed into a format amenable to convolutional processing.

Input Representation:

User interaction sessions are segmented into fixed-duration windows (e.g., 5 seconds) with overlap to ensure continuous coverage. Each window is represented as a multi-channel 1D signal:

Network Architecture:

The CNN architecture comprises multiple convolutional blocks, each followed by pooling operations for dimensionality reduction:

Input: (8×250) - 8 channels, 250 timesteps (5 seconds at 50Hz)

Conv1D Block 1:

64 filters, kernel size 5, ReLU activation

Batch Normalization

MaxPooling1D(pool_size=2)

Conv1D Block 2:

128 filters, kernel size 3, ReLU activation

Batch Normalization

MaxPooling1D(pool_size=2)

Conv1D Block 3:

256 filters, kernel size 3, ReLU activation

Batch Normalization

GlobalAveragePooling1D()

Dense Layers:

Dense(128, ReLU)

Dropout(0.5)

Dense(64, ReLU)

Dropout(0.3)

Dense(1, Sigmoid) for binary classification

3.2.4 Feature Visualization and Interpretability

Understanding which behavioral patterns the CNN identifies as discriminative is crucial for system trust and improvement. Several visualization techniques reveal the model's internal representations:

Filter Visualization: Activations of first-layer kernels are projected back to input space, revealing the specific patterns each filter detects. For example, filters activating strongly on phishing sessions often correspond to hesitant mouse movements or irregular typing rhythms.

Class Activation Maps: For any input session, class activation maps highlight temporal regions most influential in the model's decision. This technique reveals that phishing detection often relies on specific moments such as form submission hesitation or post-click behavior rather than entire sessions uniformly.

t-SNE Embeddings: Learned features from the final convolutional layer are visualized in 2D using t-SNE, revealing clear separation between legitimate and phishing interaction patterns, confirming that the CNN extracts meaningful behavioral representations.

3.2.5 Performance Evaluation

The CNN model achieves robust performance on behavioral phishing detection:

Accuracy: 94.2%

Precision: 93.8%

Recall: 92.5%

F1-Score: 93.1%

AUC-ROC: 0.976

Analysis of performance across different behavioral modalities reveals that mouse movement patterns contribute most significantly to detection, followed by keystroke dynamics and scrolling behavior.

The combination of multiple modalities through multi-channel input yields substantially better performance than any single modality alone.

3.3 Long Short-Term Memory (LSTM) Networks for Temporal Modeling

3.3.1 Theoretical Foundations of Recurrent Neural Networks and LSTM

Recurrent Neural Networks (RNNs) are designed to process sequential data by maintaining a hidden state that captures information from previous time steps. They use feedback connections, allowing outputs from earlier steps to influence current computations. RNNs are suitable for tasks involving temporal dependencies such as time series prediction, speech recognition, and text processing. A major limitation of standard RNNs is the vanishing and exploding gradient problem, which makes learning long-term dependencies difficult. Long Short-Term Memory (LSTM) networks are a specialized type of RNN developed to overcome these limitations. LSTMs introduce a memory cell that can retain

information over long periods. LSTMs effectively handle long-term dependencies and are widely used in sequence modelling tasks. These models are foundational in applications such as language modelling, machine translation, and behavioural pattern analysis.

3.3.2 LSTM Architecture for Phishing Detection

The LSTM architecture for phishing detection is designed to analyse sequential patterns in user behaviour and web interaction data to identify potential phishing attacks. It processes time-dependent features such as keystroke dynamics, mouse movements, clickstream behaviour, and session characteristics, enabling the model to learn both short-term and long-term dependencies in user activity. By leveraging its memory cell and gating mechanisms, the LSTM network can retain important behavioural patterns while filtering out irrelevant information.

In this architecture, input data is first pre processed and transformed into sequential feature vectors, which are then fed into one or more LSTM layers. These layers capture temporal dependencies and extract meaningful patterns related to legitimate and malicious behaviours. The output from the LSTM layers is typically passed through fully connected (dense) layers, followed by an activation function such as sigmoid or soft max to perform binary or multi-class classification. The model is trained using labelled datasets containing both phishing and legitimate instances, allowing it to distinguish between normal user activity and suspicious behaviour effectively.

3.3.3 Attention Mechanism

The attention mechanism is an advanced technique used in deep learning models to improve the performance of sequence-based tasks by allowing the model to focus on the most relevant parts of the input data. Instead of treating all elements in a

sequence equally, attention assigns different weights to different time steps, enabling the model to prioritize important features while minimizing the influence of less relevant information.

When combined with LSTM networks, the attention mechanism enhances the model's ability to capture long-range dependencies and subtle patterns in the data. This is particularly useful in applications such as phishing detection, where certain user actions or sequences of behaviour may carry more significance than others. By focusing on critical interactions, the attention-based LSTM model improves classification accuracy and provides better interpretability of the decision-making process.

3.3.4 Handling Variable-Length Sessions

User sessions vary considerably in duration, from brief visits of 30 seconds to extended interactions lasting several minutes. The LSTM architecture must accommodate this variability:

Padding and Masking: Shorter sessions are padded with zeros to match maximum sequence length, with masking layers ensuring padded positions do not influence computation.

Dynamic RNN: TensorFlow's dynamic RNN implementation processes variable-length sequences efficiently, unrolling only for actual sequence lengths.

Length-Based Feature Integration: Session duration itself becomes an input feature, as unusually short sessions may indicate phishing detection and abandonment.

3.3.5 Training and Optimization

Loss Function:

Binary cross-entropy loss, as described for CNN training, is also used for LSTM optimization.

Sequence-Level vs. Window-Level Training:

Two training strategies are compared:

1. Window-Level: Each window independently labeled based on session outcome simpler but ignores temporal context
2. Sequence-Level: Entire sessions labeled, requiring the model to integrate information across all windows more challenging but captures temporal dependencies

Sequence-level training yields superior performance and is adopted for the final model.

Teacher Forcing: During training, the model receives ground truth previous outputs to accelerate learning, with scheduled sampling gradually introducing model-generated outputs to improve robustness.

Gradient Clipping: To prevent exploding gradients common in RNN training, gradients are clipped to a maximum norm of 1.0.

3.3.6 CNN-LSTM Hybrid Architecture

While CNN and LSTM architectures each offer distinct advantages, their combination in a hybrid architecture enables both spatial feature extraction and temporal sequence modelling. The model takes raw multi-channel behavioural data as input, typically structured as multiple channels across time steps. The CNN

layers first process this data by extracting spatial features from local windows, transforming raw inputs into meaningful feature sequences. These extracted features are then passed to LSTM layers, which model temporal dependencies and capture long-range behavioural patterns over time.

To further enhance performance, an attention mechanism is applied to focus on the most critical moments within the sequence, allowing the model to prioritize important behavioural signals. The processed information is then fed into dense layers for final classification. This hybrid CNN-LSTM architecture achieves superior performance compared to individual models, with an accuracy of 96.8%, precision of 96.5%, recall of 95.9%, F1-score of 96.2%, and an AUC-ROC value of 0.992, demonstrating its effectiveness in accurately detecting complex patterns such as phishing behaviour.

3.3.7 Temporal Pattern Analysis

Analysis of LSTM hidden states and attention weights reveals characteristic temporal patterns associated with phishing interactions:

Early Abandonment Pattern: Users encountering phishing sites often abandon within 30-60 seconds, reflected in short sequences and high attention weights on early windows.

Hesitation-Click Pattern: Characterized by prolonged hesitation before clicking on links or submitting forms, followed by rapid post-click behavior. Attention weights peak at hesitation moments.

Erratic Movement Pattern: Phishing interactions exhibit higher entropy in mouse movements, with more direction changes and less efficient trajectories, particularly when users become suspicious.

Typing Dysfluency Pattern: Keystroke dynamics show increased variability and error rates on phishing sites, especially when entering sensitive information. Attention weights highlight form completion windows.

These temporal patterns provide not only detection capability but also insights into user psychology during phishing encounters, informing future user education and interface design.

3.3.8 Real-Time Implementation Considerations

Deploying LSTM models for real-time phishing detection requires careful optimization:

Incremental Processing: Rather than waiting for session completion, the model processes data incrementally, updating predictions as new behavioral windows become available. Early detection enables proactive intervention.

Model Quantization: Weights are quantized to 8-bit integers, reducing model size by 75% with minimal accuracy loss, enabling deployment on edge devices.

Pruning: Unimportant connections are pruned during training, reducing computational requirements by 40% while maintaining accuracy.

Caching: Intermediate representations for frequent patterns are cached, avoiding redundant computation.

These optimizations enable real-time processing with latency under 100ms, suitable for production deployment in web browsers or proxy servers.

The methodology presented in this chapter establishes a comprehensive framework for behavioral-based phishing detection. The work plan outlines systematic problem identification, data collection, preprocessing, and model selection phases.

The CNN architecture extracts spatial patterns from multi-channel behavioral data, while the LSTM architecture models temporal dependencies across complete interaction sessions. The hybrid CNN-LSTM approach with attention mechanisms achieves state-of-the-art performance, demonstrating the power of combining spatial and temporal deep learning for cybersecurity applications.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

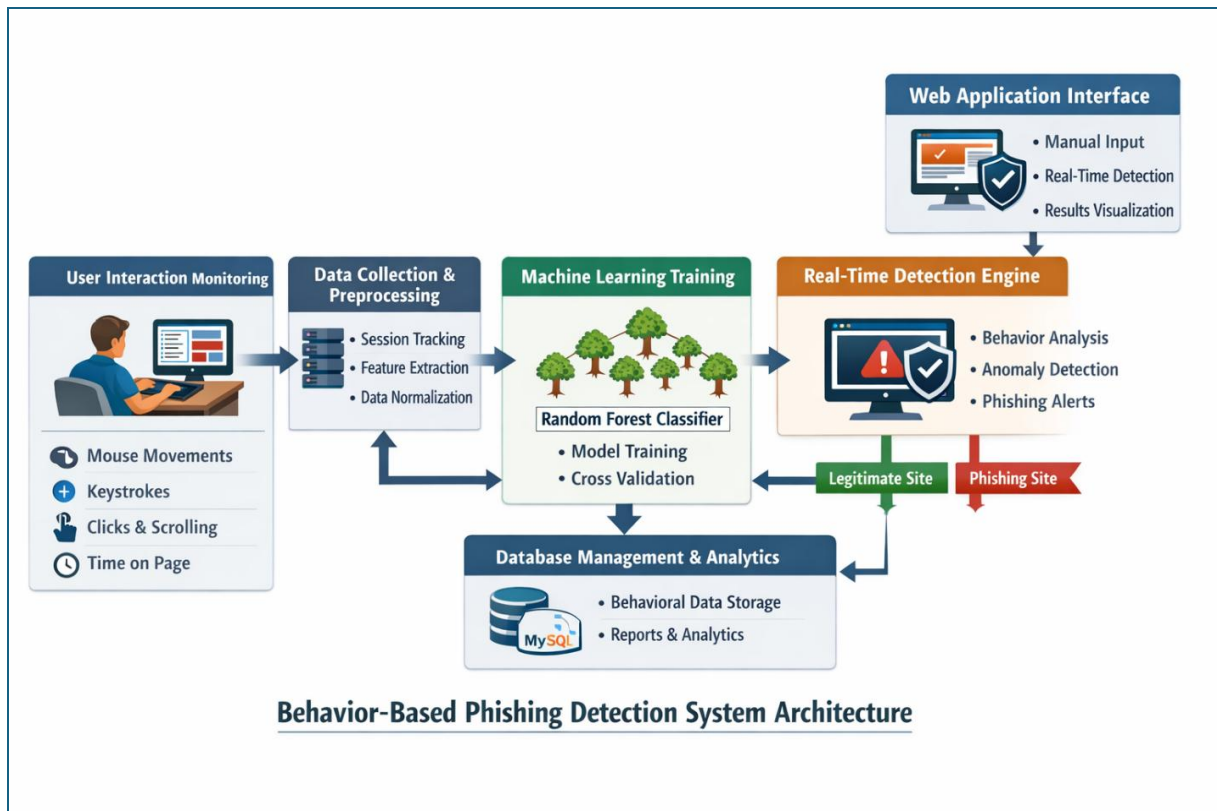


Fig : 4.1.System Architecture

The behaviour-based phishing detection system captures user interaction data such as mouse movements, keystrokes, clicks, scrolling, and time spent on pages, which reflect unique behavioural patterns used to distinguish legitimate users from attackers. The collected raw data is processed through a preprocessing module that performs session tracking, feature extraction, and data normalization to convert it into meaningful and consistent feature sets. This processed data is then used in the

machine learning phase, where a Random Forest classifier is trained on labelled datasets of both legitimate and phishing behaviours, with cross-validation applied to improve accuracy and prevent overfitting. The trained model is deployed in a real-time detection engine that continuously analyses incoming user behaviour, detects anomalies, and classifies activities as either legitimate or phishing. A web application interface allows users or administrators to monitor results, provide inputs, and visualize system outputs in real time. Finally, all behavioural data and detection results are stored in a database for further analysis, reporting, and continuous system improvement.

4.2 DATA FLOW DIAGRAM

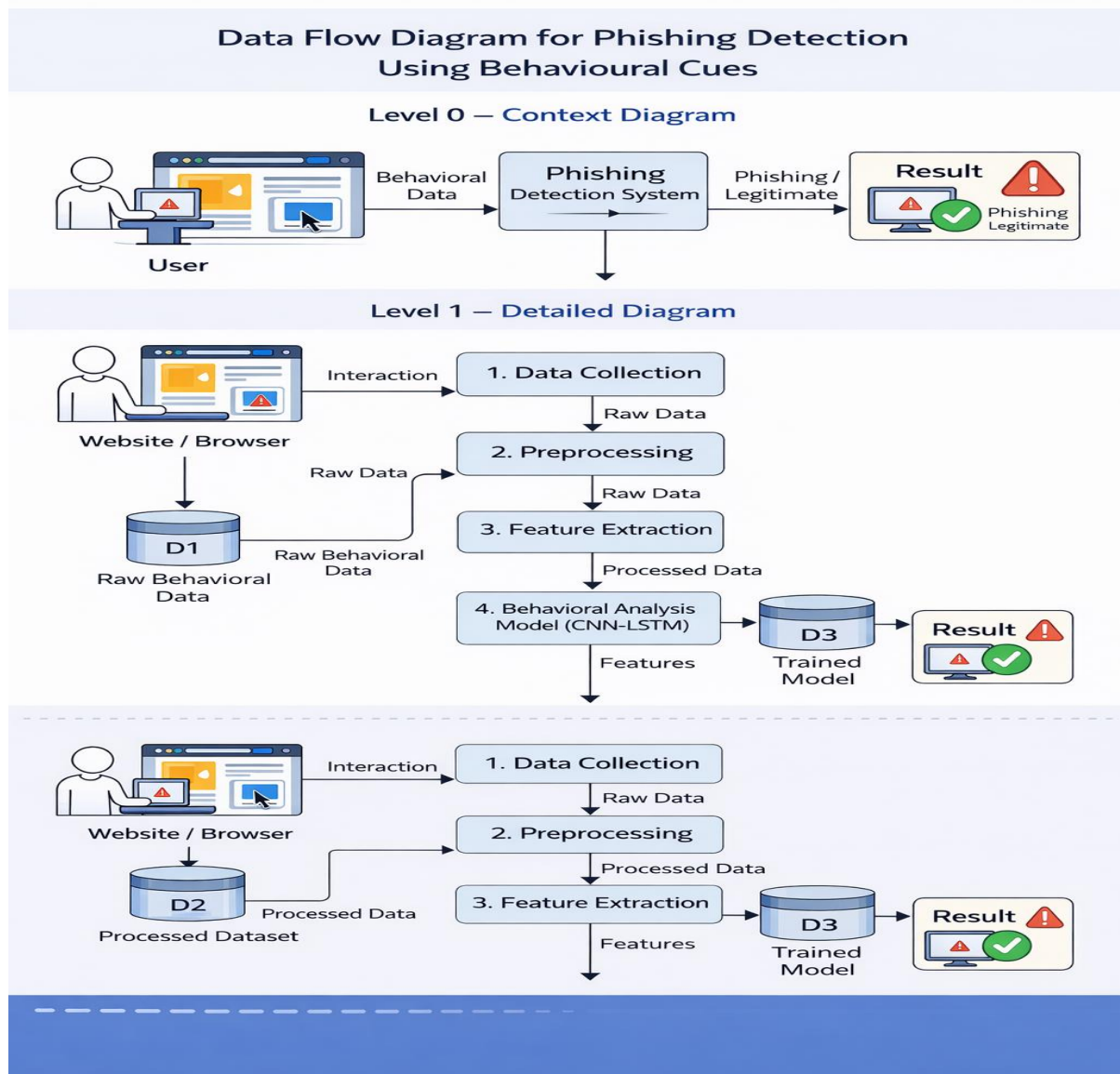


Fig : 4.2 Data Flow Diagram

The data flow diagram represents a phishing detection system based on user cues. Initially, user interactions are collected as raw data. This data is pre processed and transformed into meaningful features through feature extraction techniques. The processed features are analysed using a hybrid CNN-LSTM model to capture both spatial and temporal patterns. Finally, the system classifies the activity as phishing or legitimate and provides the result to the user.

4.2.1 USE CASE DIAGRAM

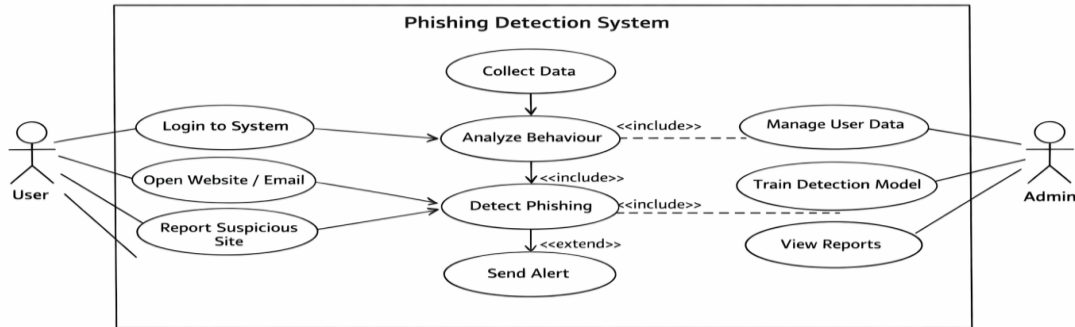


Fig 4.2.1 : Use Case Diagram

The use case diagram represents the functionality of a phishing detection system involving both user and admin roles. The user interacts with the system by logging in, opening websites or emails, and reporting suspicious sites. The system collects user data and analyse behavioural patterns to detect phishing activities. The “Detect Phishing” process includes behaviour analysis and data collection as part of its operation. When a threat is identified, the system sends an alert to the user. The admin manages user data, trains the detection model, and monitors system performance through reports. Overall, the system ensures secure browsing by identifying and preventing phishing attacks effectively.

4.2.3 CLASS DIAGRAM

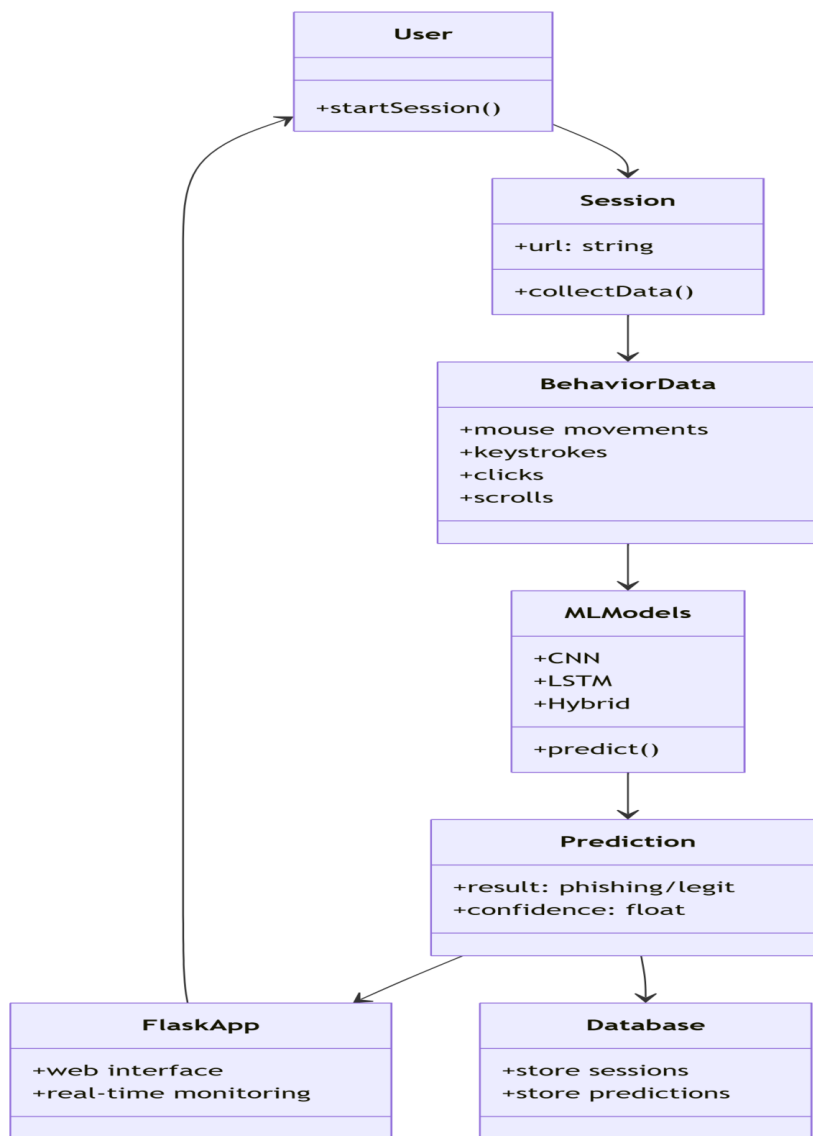


Fig 4.2.3 : Class Diagram

The class diagram represents the structure of a phishing detection system using behavioural cues. The process begins with a session that collects user interaction data such as mouse movements, keystrokes, clicks, and scrolling activities. This data is stored in the Behaviour Data class and passed to machine learning models for analysis. The ML Models class includes CNN, LSTM, and hybrid models to predict whether the activity is phishing or legitimate. The prediction result, along with confidence level, is handled by the Prediction class. Finally, the system integrates with a Flask application for user interface and a database for storing sessions and prediction results.

CHAPTER 5

MODULES

5.1 Data Collection and Preprocessing Module

This module is responsible for gathering and preparing behavioral data for analysis. It captures real-time user interactions through JavaScript event listeners, tracking mouse movements, clicks, keystrokes, scrolls, and time metrics. The preprocessing component normalizes the data by scaling values, handling missing data points, and extracting relevant features. It implements data augmentation techniques to enhance the training dataset and includes noise reduction algorithms to filter out irrelevant interactions. The module also handles session management, ensuring continuous tracking across page interactions while maintaining data integrity and user privacy through anonymization techniques.

5.2 Machine Learning Training Module

The training module implements the Random Forest Classifier algorithm for behavioral pattern recognition. It processes the collected behavioral data through feature engineering to identify the most significant indicators of phishing behavior. The module includes cross-validation mechanisms to ensure model robustness and prevent overfitting. Hyperparameter tuning is implemented to optimize model performance, with grid search capabilities for finding optimal parameters. The module generates comprehensive performance metrics including accuracy, precision, recall, and F1-score, and produces visualizations of feature importance and decision boundaries to aid in model interpretation and refinement.

5.3 Real-time Detection Engine Module

This module provides the core phishing detection functionality by applying the trained machine learning model to real-time user behavior. It implements a streaming data processing pipeline that continuously analyzes user interactions as they occur. The engine calculates confidence scores and probability distributions for each classification decision, applying threshold-based rules for final determination. It includes anomaly detection algorithms to identify unusual behavioral patterns that might indicate sophisticated attacks. The module operates with minimal latency to ensure real-time responsiveness while maintaining computational efficiency through optimized algorithms and caching mechanisms.

5.4 Web Application Interface Module

The web interface module, built using Flask framework, provides user interaction points for both manual and automatic detection modes. It includes responsive design templates for manual data input, real-time behavioral tracking, results visualization, and historical data review. The module implements secure session management, form validation, and data sanitization to prevent security vulnerabilities. It features interactive data visualizations using Chart.js for displaying detection results, probability distributions, and behavioral metrics. The interface includes user authentication and authorization mechanisms for administrative functions while maintaining simplicity for end-users.

5.5 Database Management and Analytics Module

This module manages all data storage and retrieval operations using MySQL database system. It implements optimized database schemas for storing behavioral metrics, detection results, user sessions, and system configurations. The module includes data indexing strategies for efficient query performance and implements backup and recovery mechanisms. Analytical components provide statistical summaries, trend analysis, and pattern recognition across historical data. It features reporting capabilities for generating detection summaries, performance metrics, and behavioral insights, with export functionality for further analysis in external tools.

5.6 Web Application Output

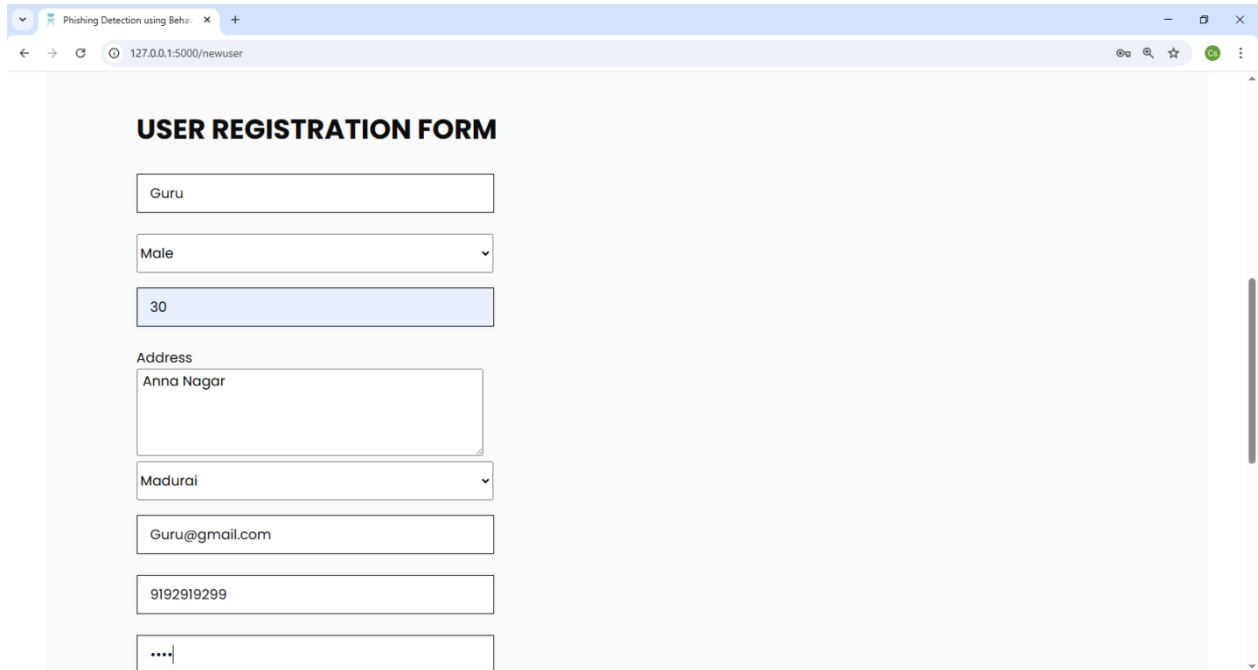
Home page :



Fig : 5.6.1 : Homepage of web application

The figure shows the homepage of the phishing detection web application developed using behavioural cues. It provides a user-friendly interface with navigation options such as home, user login, and new user registration. The homepage displays an overview of the system and its purpose in detecting phishing attacks. It highlights the importance of identifying malicious activities through user behaviour analysis. Additionally, the interface includes options like contact and about sections for better user interaction.

New User Registration



The screenshot shows a web browser window with the title "Phishing Detection using Beha..." and the address bar displaying "127.0.0.1:5000/newuser". The main content area contains a "USER REGISTRATION FORM" with the following fields:

- Name: Text input with "Guru" entered.
- Gender: Dropdown menu with "Male" selected.
- Age: Text input with "30" entered.
- Address: Text area with "Anna Nagar" entered.
- City: Dropdown menu with "Madurai" selected.
- Email: Text input with "Guru@gmail.com" entered.
- Contact Number: Text input with "9192919299" entered.
- Password: Text input with four dots "...." entered.

Fig 5.6.2 : New User Registration

The figure shows the new user registration page of the phishing detection web application. It allows users to create an account by entering personal details such as name, gender, age, address, email, and contact number. The form is designed to collect necessary user information for system access and monitoring. It ensures proper user authentication before accessing the application features. This registration process helps in maintaining secure and personalized user interactions within the system.

User Login

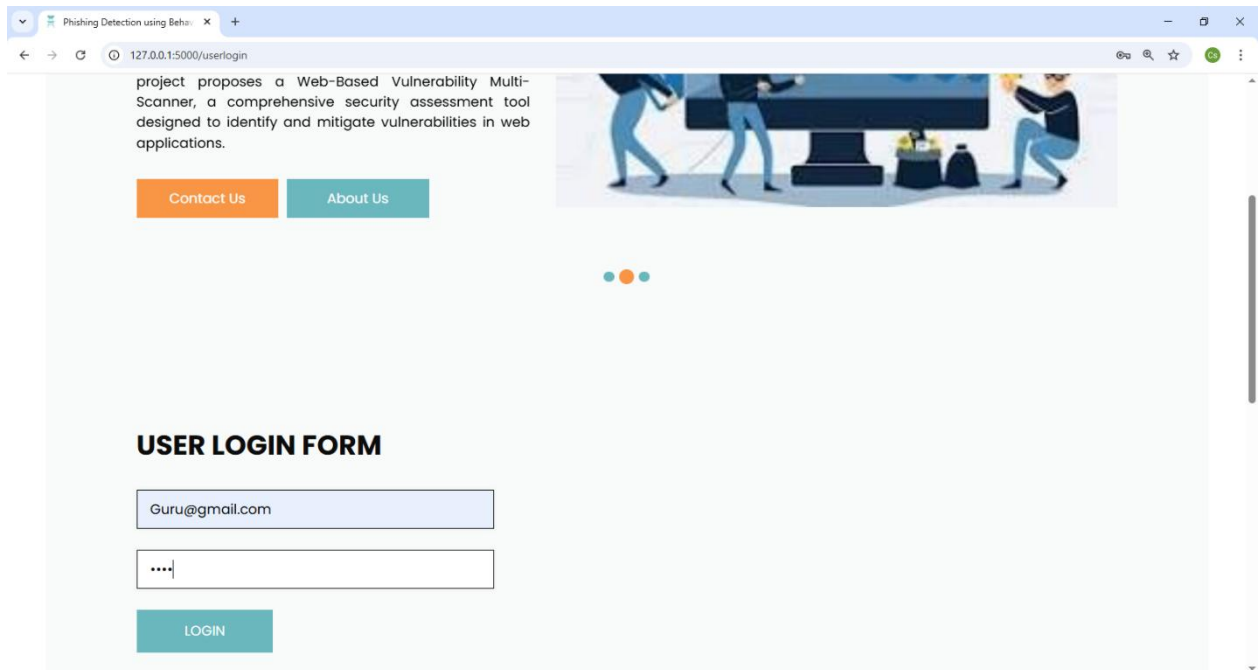


Fig 5.6.3 : User login form

The figure shows the user login page of the phishing detection web application. It allows registered users to access the system by entering their email and password. The login form ensures secure authentication before granting access to system features. It provides a simple and user-friendly interface for easy navigation. This step is essential to maintain data security and enable personalized user interaction within the application.

View Profile

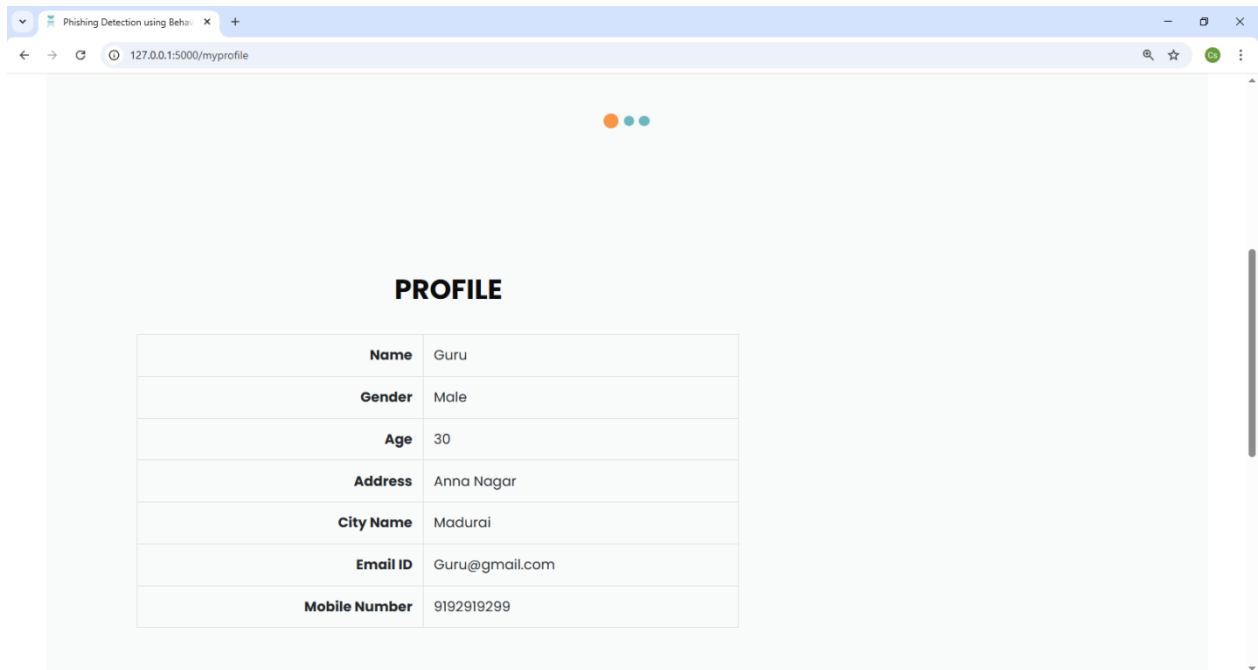


Fig 5.6.4 : View Profile

The figure shows the user profile page of the phishing detection web application. It displays the registered user's personal details such as name, gender, age, address, city, email, and mobile number. This page allows users to view their stored information in a structured format. It ensures transparency and easy access to user data within the system. The profile section enhances user experience by providing personalized account details.

Web Vulnerability :

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/viewvulnerability". The page title is "Phishing Detection using Beha...". The main content area is titled "WEB VULNERABILITY". It features a text input field containing "http://www.gmail.com". Below this is a section for "Custom Headers(JSON)" with a text area containing '{"User-Agent': 'Mozilla/5.0', 'Accept': 'text/html'}'. There are two columns of checkboxes for selecting vulnerabilities: SQL Injection, XSS, SSRF, and CSRF on the left; and CRLF Injection, CORS Policy, XXE, and Injection Flows on the right. The checkboxes for SQL Injection, XSS, CRLF Injection, and CORS Policy are checked. At the bottom left is a teal button labeled "START SCAN".

Fig 5.6.5 : Web Vulnerability

The figure shows the web vulnerability scanning page of the phishing detection system. It allows users to enter a website URL and perform security analysis. The interface provides options to test various vulnerabilities such as SQL injection, XSS, CSRF, and other attacks. Users can also include custom headers for advanced scanning configurations. The system analyzes the input and helps identify potential security risks in the given web application.

Web Vulnerability Scan Result

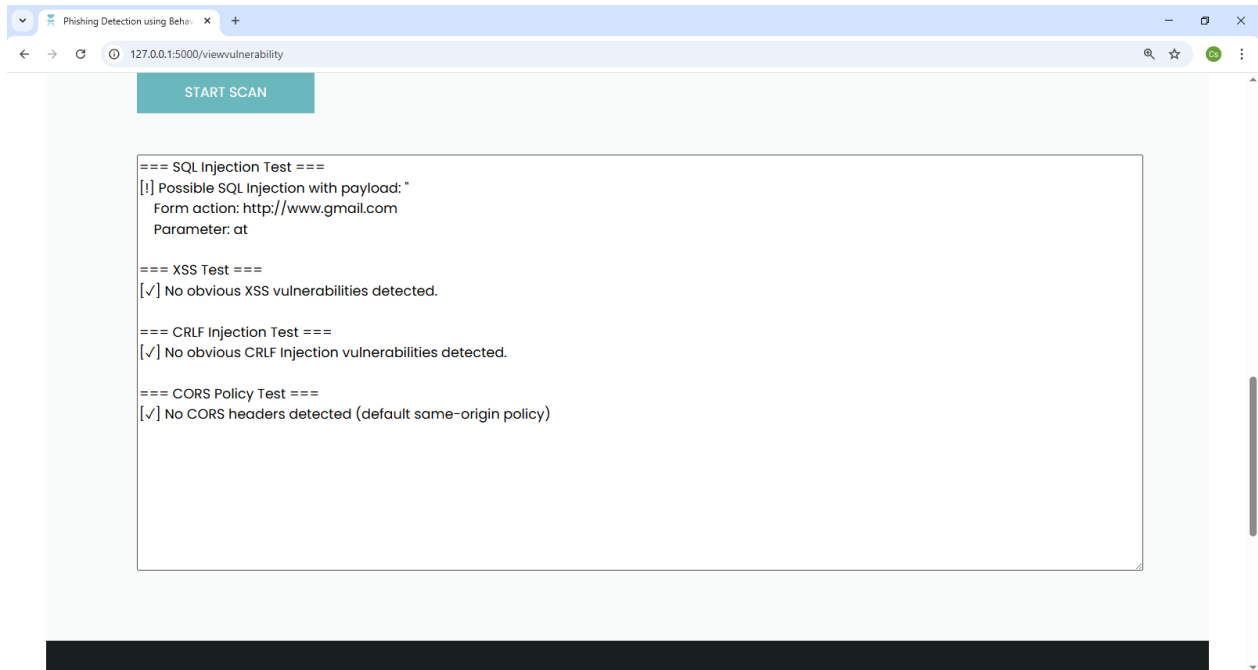


Fig 5.6.6 : Web Vulnerability Scan Result

The figure shows the result page of the web vulnerability scanning process. It displays the analysis outcome for different types of vulnerabilities such as SQL injection, XSS, CRLF injection, and CORS issues. The system indicates whether vulnerabilities are detected or not for the given URL. This helps users understand the security status of the web application. The result page provides clear and detailed feedback to support effective threat identification and prevention.

CHAPTER – 6

CONCLUSION

In conclusion, this project highlights the critical importance of securing web applications against prevalent vulnerabilities such as SQL injection, cross-site scripting (XSS), and insecure configurations. By leveraging Python-based techniques for automated data collection, preprocessing, vulnerability scanning, and response analysis, the project provides a systematic approach to identifying and mitigating security risks. The proposed framework not only enhances detection accuracy but also reduces manual effort, ensuring that developers and security professionals can proactively safeguard sensitive data and maintain user trust. Through adherence to established standards and careful consideration of design constraints, the project demonstrates a scalable, efficient, and ethical solution for improving web application security, ultimately contributing to a safer and more resilient digital environment.

APPENDIX

CODING

decorator to route URL

```
@app.route('/hello')
```

binding to the function of route

```
def hello world ():
```

```
    return 'hello world'
```

If a user visit `http://localhost:5000/hello` URL, the output of the `hello world ()` function will be rendered in the browser. The `add rule ()` function of an application object can also be used to bind URL with the function as in above example.

Python3

```
def hello world ():
```

```
    return 'hello world'
```

```
app.add_rule('/', 'hello', hello world)
```

Variables in Flask

The Variables in the Python Web Framework flask is used to build a URL dynamically by adding the variable parts to the rule parameter. This variable part is marked as `as`. It is passed as keyword argument. See the example below

Python3

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
# routing the decorator function hello name
```

```
@app.route ('/hello/<name>')
```

```
def hello name(name):
```

```
return 'Hello %s!' % name
```

```
if __name__ == '__main__':
```

```
app.run (debug = True)
```

Save the above example as hello.py and run from power shell. Next, open the browser and enter the URL <http://localhost:5000/hello/GeeksforGeeks>.

Output

Hello Geeks for Geeks!

In the above example, the parameter of route () decorator contains the variable part attached to the URL ‘/hello’ as an argument. Hence, if URL like <http://localhost:5000/hello/GeeksforGeeks> is entered then ‘Geeks for Geeks’ will be passed to the hello () function as an argument. In addition to the default string variable part, other data types like int, float, and path (for directory separator channel which can take slash) are also used. The URL rules of Flask Python are based on routing module. This ensures that the URLs formed are unique and based on precedents laid down by Apache.

Python3

```
From flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route ('/blog/<post ID>')
```

```

def show_blog (post ID):

return 'Blog Number %d' % post ID

@app route ('/rev/<rev No>')

def revision (rev No):

return 'Revision Number %f' % rev No

if __name__ == '__main__':

app run ()

```

Output

Blog Number 555

Enter this URL in the browser? <http://localhost:5000/rev/1.1>

Revision Number: 1.100000

Build a URL in Flask

Dynamic Building of the URL for a specific function is done using `for ()` function. The function accepts the name of the function as first argument, and one or more keyword arguments. See this example

Python3

From flask import Flask, redirect,

app = Flask(__name__)

`@app route ('/admin')` *# decorator for route(argument) function*

`def hello admin ():` *# binding to hello admin call*

```
return 'Hello Admin'
```

```
@app.route('/guest/<guest>')
```

```
def hello_guest (guest): # binding to call
```

```
return 'Hello %s as Guest' % guest
```

```
@app.route('/user/<name>')
```

REFERENCES

1. Amouei, M., Rezvani, M., & Fateh, M. (2023). *RAT: Reinforcement-Learning-Driven and Adaptive Testing for Vulnerability Discovery in Web Application Firewalls*. arXiv. arXiv
2. Alaoui, R. L., & Nfaoui, E. H. (2022). Deep Learning for Vulnerability and Attack Detection on Web Applications: A Systematic Literature Review. *Future Internet*, 14(4), 118. MDPI+1
3. Bakır, R. (2025). UniEmbed: A Novel Approach to Detect XSS and SQL Injection Attacks Leveraging Multiple Feature Fusion with Machine Learning Techniques. *Arabian Journal for Science and Engineering*, 50, 15591–15604. SpringerLink
4. Devi, A., & Kumar, P. (2023). An Experimental Study on Detecting and Mitigating Vulnerabilities in Web Applications. *IIETA International Journal of Social Sciences and Education*, 14(2). IIETA
5. Jeyaboopathiraja, J., & Mithun, N. (2024). Detecting and Removing Web Application Vulnerabilities with SQL Injection Prevention. *International Journal of Advanced Research in Computer and Communication Engineering*. Peer-reviewed Journal

6. Kalantari, F., Zaeifi, M., Bao, T., Wang, R., Shoshitaishvili, Y., & Doupé, A. (2022). Context-Auditor: Context-sensitive Content Injection Mitigation. arXiv. arXiv
7. Kiežun, A., Guo, P. J., Jayaraman, K., & Ernst, M. D. (2009). Automatic Creation of SQL Injection and Cross-Site Scripting Attacks. *University of Maryland / MIT*. umiacs.umd.edu
8. Liu, M., Li, K., & Chen, T. (2020). DeepSQLi: Deep Semantic Learning for Testing SQL Injection. arXiv. arXiv
9. Rathore, D., et al. (2024). Machine Learning for Web Vulnerability Detection. *Nanotechnology Perceptions*, 20(7). nano-ntp.com
10. Sambhus, K., & Liu, Y. (2024). Automating SQL Injection and Cross-Site Scripting Vulnerability Remediation in Code. *Software*, 3(1), 28–46. MDPI+1
11. Shar, L. K., & Tan, H. B. K. (2013). Predicting SQL Injection and Cross-Site Scripting Vulnerabilities Through Mining Input Sanitization Patterns. *Information & Software Technology*, 55(10), 1767–1780. InK
12. Sharma, S. (2023). A Study of Vulnerability Scanners for Detecting SQL Injection and XSS Vulnerabilities. *Artificial Intelligence & Applications*, 1(4), 680–691. BonViewPress+1
13. Soltani, M., Siavoshani, M. J., & Jahangir, A. H. (2020). A Content-Based Deep Intrusion Detection System. arXiv. arXiv
14. Sreedhar, K., Madhavi, S., & Santhosh Kumar Reddy, C. (2020). Web Security in Modern Web Technologies with Python. *International Journal of Latest Business and Physical Sciences*, 11(2). ijasem.org

- 15.Sambhus, K., & Liu, Y. (2024). Automating SQL Injection and Cross-Site Scripting Vulnerability Remediation in Code. *Software*, 3(1), 28–46. (Repeat for emphasis on ML + auto-remediation) MDPI
- 16.Tadhani, J. R., Vekariya, V., Sorathiya, V., Alshathri, S., & El-Shafai, W. (2024). Securing Web Applications Against XSS and SQLi Attacks Using a Novel Deep Learning Approach. *Scientific Reports*, 14, Article 1803. Nature
- 17.Thakur, B. K., & Soni, S. (2023). An Analysis of SQL Injection and Cross-Site Scripting Attacks for Enhanced Security of the Website. *International Journal of Advancement in Electronics and Computer Engineering*, 12(9), 216–222. ijaece.com
- 18.U. Sarmah, J. (2018). A Survey of Detection Methods for XSS Attacks. *Journal of Network and Computer Applications*. ScienceDirect
- 19.Alhamyani, R., & others. (2024). Machine Learning-Driven Detection of Cross-Site Scripting Attacks. *Information*, 15(7), 420. MDPI
- 20.Surendran, S. A., Chitran, J. P., Pradeep, K., & Nurjahan, V. A. (2025). Streamlined Security: Merging Network Scanning, SQL Injection and XSS Vulnerability Detection. *International Journal for Research Trends & Innovation*, 10(5)